Module 1 — Laboratory: Your First Project on GitHub

Vistula University · IT Fundamentals · Laboratory assignment

Estimated time: 60–90 minutes **Work mode:** in pairs (you and the person sitting next to you)

Prerequisites

Before you start, check that you have:

- A **GitHub account** — use your academic (university) account. If you do not have one, register at https://github.com before the class.
- **Git installed** on your computer. Check it: open a terminal and type `git --version`. If you see a version number, all is good.
- A text editor (for example Visual Studio Code or Notepad).

Each student works on **their own academic GitHub account**. You will review each other's work at the end, so do not do the tasks for your partner.

Part 1 — Create a Repository and Clone It

1. Log in to GitHub with your own account.
2. Click the **+** icon (top right) and choose **New repository**.
3. Name the repository `project-website`. Add a short description, for example "My first project page for IT Fundamentals". Choose **Public**. Tick **Add a README file**. Click **Create repository**.
4. Now copy the project to your computer. In the terminal, go to a folder where you keep your school work, then type:

```
git clone https://github.com/YOUR-NAME/project-website.git
```

This copies the repository from GitHub to your computer. Replace `YOUR-NAME` with your GitHub username.

5. Go into the new folder:

```
cd project-website
```

This moves the terminal into your project folder.

Part 2 — Add Your Project Page

6. Create a new file called `index.html` in the `project-website` folder. It must describe an **imaginary software project**. It does not need JavaScript — only simple HTML. It must contain: - a title of the project, - a short description (2–3 sentences: what the project does and for whom), - the project members (you and your partner — use your names), - a list of at least three proposed features (ideas for the future).

You can use this template and change the text:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>StudyBuddy</title>
</head>
<body>
  <h1>StudyBuddy</h1>
  <p>StudyBuddy is a simple web app that helps students plan their study week.
     It shows tasks, deadlines and free time in one place.</p>
  <h2>Project members</h2>
  <ul>
    <li>Anna Kowalska</li>
    <li>Piotr Nowak</li>
  </ul>
  <h2>Proposed features</h2>
  <ul>
    <li>A weekly study plan</li>
    <li>Reminders before deadlines</li>
    <li>A shared task list for group work</li>
  </ul>
</body>
</html>
```

Part 3 — Commit and Push

7. Check what Git sees:

```
git status
```

This shows which files are new or changed. You should see `index.html` in red (not yet staged).

8. Stage and commit your work:

```
git add index.html
```

This puts `index.html` into the staging area.

```
git commit -m "Add project page with description and features"
```

This saves the staged changes into the history with a short message.

9. Send your commit to GitHub:

```
git push
```

This sends your commits from your computer to GitHub. Open your repository in the browser — `index.html` should be there.

Part 4 — Feature Branch and Pull Request

Now practise the safe workflow from the theory: never work directly on `main`.

10. Create a new branch and switch to it:

```
git checkout -b feature-badges
```

This creates a branch called `feature-badges` and switches to it in one step.

11. On this branch, make a small change: open `index.html` and add one new feature to the list (for example `<li>A dark mode button</li>`).
12. Commit and push the branch:

```
git add index.html
```

Stages the changed file.

```
git commit -m "Add dark mode to the feature list"
```

Saves the change on the `feature-badges` branch.

```
git push -u origin feature-badges
```

Sends the new branch to GitHub. The `-u` option connects your local branch to the GitHub branch.

13. Open the pull request:
    - Go to your repository on GitHub. You will see a yellow banner: **"Compare & pull request"**. Click it.
    - Check that the pull request goes **from** `feature-badges` **into** `main`.
    - Write a title (for example "Add dark mode feature") and click **Create pull request**.

Part 5 — Review Your Partner's Pull Request

Now swap roles with your partner. Your partner should do Parts 1–4 on their own repository, and you review their pull request (and then your partner reviews yours).

14. Open your partner's repository on GitHub and go to the **Pull requests** tab.
15. Open their pull request. Click the **Files changed** tab and read the changes.
16. Go back to **Conversation**, click **Review changes**, choose **Approve**, and write one kind, useful comment (for example "Looks good, clear list of features"). Submit the review.
17. Click **Merge pull request**, then **Confirm merge**. You can now click **Delete branch** to remove the feature branch on GitHub.

Part 6 — Publish the Page with GitHub Pages

18. In your repository on GitHub, open **Settings** → **Pages**.
19. Under **Build and deployment**, choose **Deploy from a branch**. Select the branch `main` and the folder `/ (root)`. Click **Save**.
20. Wait 1–2 minutes and refresh the page. A green box shows your website address, like:

```
https://YOUR-NAME.github.io/project-website/
```

21. Open the address in your browser. Your project page is now live on the internet!

Expected Deliverables

At the end of the class, each student must have:

- A public GitHub repository `project-website` with an `index.html` page (title, description, members, feature list).
- At least two commits on `main` and one commit on a feature branch.
- One merged pull request with an approving review from your partner.
- A working GitHub Pages website for the repository.

Submission Instructions

- Submit through the course platform on the assignment page (Assignments section): paste your GitHub repository URL / deployed app URL / PR URL as required: 1. the URL of your repository, for example `https://github.com/YOUR-NAME/project-website`, 2. the URL of your GitHub Pages site, for example `https://YOUR-NAME.github.io/project-website/`.
- Include **both partners' names and student IDs** in your submission. Your partner's approved review on your pull request is part of the grade, so make sure the review is done before you submit.